

9. CONCURRENCY CONTROL

Lock-Based Protocols: One way to ensure serializability is to require that data items must be accessed by x'actions in a mutually exclusive manner. That is, when one x'action is modifying a data item, no other x'action can access that data item.

The most common way to implement this requirement is to allow a x'action to access a data item only if it is currently holding a lock on that data.

There are two modes in which a data may be locked.

(i) **Shared:** If x'action T_i has obtained a shared lock on data item Q, then T_i is allowed to perform read(Q) but can not perform write (Q).

(ii) **Exclusive:** If x'action T_i has obtained an exclusive lock on data item Q , then T_i can perform both read (Q) & write (Q).

Compatible lock modes: Let A & B represent arbitrary lock modes. Suppose x'action T_i request lock mode A on data item Q on which x'action T_j currently holds lock mode B. If x'action T_i can be granted a lock on Q immediately then mode A is said to be compatible with mode B.

	S	X
S	True	False
X	False	False

s-shared lock
x-exclusive lock

Lock-compatibility matrix

A x'action is required to issue the following request in order to obtain a lock, on data item Q.

Shared lock : lock_s(Q)

exclusive lock : lock_x(Q)

A x'action can release the lock on data item Q by issuing. unlock(Q)

For e.g.

To: lock_x(A)

T1: lock_s(A)

```
read(A)
```

```
read(A)
```

$A := A - 500$

unlock (A)

write (A)

lock $s(B)$

unlock (A)

read(B)

lock-x (B)

unlock (B)

read(B)

```
display(A+B)
```

B:B + 500

```
write(B)
```

unlock(B)

If x'actions release locks too early, system may enter incorrect state.

To	T ₁
lock_x(A) read(A) A:=A - 500 write(A) unlock(A)	lock_s(A) read(A) lock_s(B) read(B) display(A+B) unlock(B)

This schedule generate incorrect value of $A + B$.

If x'actions donot unlock data items before requesting locks o other data items, undesirable situation arises where no x'actions can proceed.

To	T ₁
lock_x(A) read(A) A:=A - 500 write(A) lock_x(B)	 lock_s(B) read(B) lock_s(A)

Now both the x'actions can not proceed.

It is required that each x'action follow a set of rules, called a locking protocol, indicating when a x'action may lock & unlock data items.

Two phase locking (2PL) protocol:

It ensures serializability & requires that each x'action issue lock & unlock requests in two phases.

(i) **growing phase:** A x'action may obtain locks, but may not release any lock.

(ii) **shrinking phase:** A x'action may release locks, but may not obtain any new locks.

Initially, a x'action is in growing phase. The x'action acquires locks as needed. Once the x'action releases a lock, it enters the shrinking phase & it can not issue any more lock requests.

For e.g.

lock_s(A)	This x'action is two phase.
read(A)	All unlock instruction need not appear
lock_s(B)	only at the end
read(B)	Say, unlock(A) can be shifted just after
display(A+B)	lock_s(B)
unlock(A)	
unlock(B)	

The 2PL ensures conflict serializability. The point in the schedule where the x'action has obtained its final lock (end of growing phase), is called the lock point of the x'action.

Now all x'actions can be ordered according to their lock points & this ordering is serializability ordering for the x'actions.

Two phase locking does not ensure freedom from dead lock.

For e.g.

To	T ₁
lock_x(B) read(B) B:=B - 500 write(B) lock_x(A)	 lock_s(A) read(A) lock_s(B)

These two x'actions are two phase but they are dead locked.

Cascading rollback may occur under 2P.

For e.g.

To	T ₁	T ₂
lock_x (A) read (A) lock_s(B) read(B) unlock(A)	lock_x(A) read(A) write(A) unlock(A)	

To	T ₁	T ₂
.	.	
.	.	
.	.	lock_s(A)
.	.	read(A)
		.
		.
		.

Here each x'action follows 2PL but failure of To after read (A) of T₂ leads to cascading rollback of T₁ & T₂.

Strict 2 PL: Cascading rollbacks can be avoided by modification of 2PL called strict 2PL, where all x'actions are not only required to do locking in 2 phase, but all exclusive-mode locks must be hold by the x'action until it commits. This requirement ensure that any data written by an uncommitted x'action are locked until x'action commits, preventing any other x'action from reading the data.

Rigorous 2PL: Another variant of 2PL, which requires that all locks must be held until the x'action commits.

Time stamp based protocols: [May-07]

Time stamp: Each x'action T_i is assigned a unique fixed time-stamp denoted by T_s(T_i); when T_i starts execution. If a new x'action T_j enters the system then T_s(T_i) < T_s(T_j).

There are two simple methods to implement time stamps: (i) use of the system clock as the time stamp (ii) use a logical counter that is incremented every time a x'action enters the system.

The time stamps of the x'actions determine the serializability order, that is, if T_s(T_i) < T_s(T_j) then system must ensure that the schedule is equivalent to a serial schedule where x'action T_i appears before T_j.

To implement this scheme, each data item Q is associated with two time stamps.

(i) W-time stamp (Q) denotes the time stamps of the youngest x'actino that executed write (Q).

(ii) R-time stamp (Q) denotes timestamp of the youngest x'action that executed read (Q).

These timestamps are updated whenever read (Q) or write (Q) is executed.

Time stamp ordering protocol: It ensures that any conflicting read & write instructions are executed in timestamp order. It operates as follows:

(i) Suppose x'action T_i issues read (Q)

(a) if T_s(T_i) < W-timestamp(Q), then T_i is trying to read a value of Q that was written by a x'action younger to T_i. System will reject read opⁿ & T_i is rolled back.

(b) if T_s(T_i) ≥ w.timestamp(Q), then read opⁿ is execute & R-timestamp (Q) updated.

(ii) Suppose x'action T_i issues write (Q).

(a) If T_s(T_i) < R-timestramp(Q), then T_i is producing a value that was needed previously system rejects the write opⁿ & T_i is rolled back.

(b) If $T_s(T_i) < W\text{-timestamp}(Q)$, then T_i is trying to write over the value that was written by a x'action younger to T_i . System rejects the write op^n & T_i is rolled back.

(c) Otherwise write op^n is executed & $w\text{-timestamp}(Q)$ is updated.

If a x'action T_i is rollback by this protocol, the system assign it a new timestamp & restarts it.

For e.g.

T_0	T_1
read(B) write(B)	read(B) read(A)
read(B) write(A)	

This schedule is not allowed.

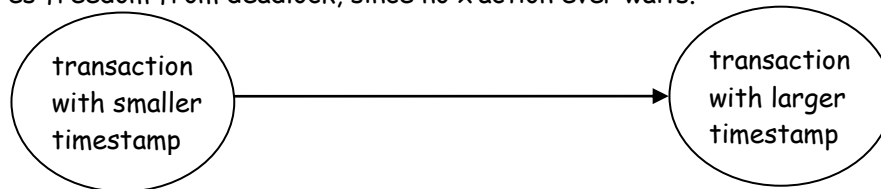
For e.g.

T_0	T_1
read(B) write(B)	read(B)
read(A) write(A)	read(A)

This schedule is allowed.

Advantages: The limestamp ordering protocol ensures conflict serializability, because conflicting op^n are processed in timestamp order.

This protocol also ensures freedom from deadlock, since no x'action ever waits.



Disadvantages:

There is a possibility of starvation of long x'actions if a sequence of conflicting short x'actions causes repeated restarting of the long x'action.

Protocol can generate schedules that are not recoverable.

[Q: Write notes on: Timestamp ordering protocol. May08]

Validation-Based Protocol [May-07]

Execution of transaction T_i is done in three phases.

1. Read and execution phase: Transaction T_i writes only to temporary local variables.
2. Validation phase: Transaction T_i performs a "validation test" to determine if local variables can be written without violating serializability.
3. Write phase: If T_i is validated, the updates are applied to the database; otherwise, T_i is rolled back.

The three phases of concurrently executing transactions can be interleaved, but each transaction must go through the three phases in that order.

Assume for simplicity that the validation and write phase occur together, atomically and serially

I.e., only one transaction executes validation/write at a time.

- Also called as **optimistic concurrency control** since transaction executes fully in the hope that all will go well during validation.
- Each transaction T_i has 3 timestamps
 - $Start(T_i)$: the time when T_i started its execution

- Validation(T_i): the time when T_i entered its validation phase
- Finish(T_i): the time when T_i finished its write phase
- Serializability order is determined by timestamp given at validation time, to increase concurrency.
 - Thus $TS(T_i)$ is given the value of Validation(T_i).
- This protocol is useful and gives greater degree of concurrency if probability of conflicts is low.
 - because the serializability order is not pre-decided, and
 - relatively few transactions will have to be rolled back.

[Q: Differentiate between the various concurrency control schemes. May08]

[Q: Explain Concurrency Control in Database System with the help of any two protocols. May08]

Deadlock Handling [May-08, May-07, Dec-06]

Deadlock: Under concurrent environment there may be a set of x'actions such that each x'action in the set has locked certain data items & is waiting for some other x'action to release the lock on certain data item. Under this situation no x'action can proceed & system is said to enter a deadlock state.

There two basic approaches to deal with deadlocks.

(i) **dead lock prevention:** It ensures that the system will never enter deadlock state.

(ii) **deadlock detection & recovery:** It allows the system to enter dead lock state, detects it & then recovers the system from deadlock state.

Prevention is used if the probability of occurring deadlock is high, otherwise detection & recovery are more efficient.

Deadlock Prevention: There are many approaches:

(i) Each x'action locks all its data items before it begins execution. Moreover, either all are locked in one step or none are locked.

Limitation: It is difficult to predict what data items are needed prior to execution of the x'action. Data items are underutilized.

(ii) It is based on preemption & rollback.

In preemption, when a x'action T_2 requests a lock that x'action T_1 holds, the lock granted to T_1 may be preempted by rolling back T_1 & granting it to T_2 .

To control preemption, timestamps are assigned to all x'actions & are used only to decide whether a x'action should wait or rollback.

If x'action is rolled back, it retains its old timestamp when restarted.

There are two methods in this approach:

(a) **Wait-die:** It is non-preemptive method. When a x'action T_i requests a data item currently held by T_j , then T_i is allowed to wait only if it is older than T_j . Otherwise, T_i is rolled back.

x'actions	:	T_1	T_2	T_3
timestamps:		(5)	(10)	(15)

If T_1 requests a data item held by T_2 then, T_1 waits. If T_3 requests data item held by T_2 then T_3 is rolled back.

(b) **Would-wait:** This is preemptive method when x'action T_i requests a data item currently held by x'action T_j then T_i is allowed to wait only if it is younger than T_j , otherwise, T_j is rolled back, (wounded by T_i).

x'actions	:	T_1	T_2	T_3
timestamps:		(5)	(10)	(15)

If T_1 requests a data item held by T_2 then data item is preempted from T_2 & T_2 is rolled back. If T_3 requests data item held by T_2 then T_3 must wait.

Whenever the system rolls back x'actions it is important to ensure that there is no starvation - i.e. no x'action gets rolled back repeatedly & is never allowed to proceed.

Both above methods avoid starvation. In both the methods the oldest x'action is never rolled back. Since timestamps always increase & since x'actions are not assigned a new timestamp when it is rolled back, a x'action that is rolled back will eventually become oldest & not rolled back again.

Differences: In "waitdie" method the older x'action must wait for younger to release lock. Thus, as x'action becomes older it tends to wait more. Whereas in "wound-wait" method older x'action never waits.

In the "wait-die" method, if a x'action T_i is rolledback because it requested data item held by T_j , then T_i may reissue the same data item when it is restarted. If the data item is still held by T_j the T_i is rolled back again. So T_i may be rolled back many times before it can proceed whereas in "wound-wait" method if T_i is rolled back because of T_j , when T_i is restored & requests the data now being held & T_j then T_i waits.

So, there may be fewer rollbacks in the "wound-wait" method.

Deadlock detection & recovery:

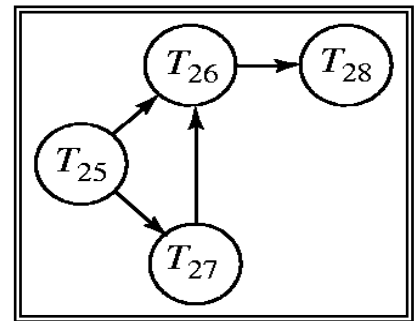
An algorithm that examines the state of the system is invoked periodically to determine whether a deadlock has occurred. If it has, then the system attempts to recover from the dead lock.

Deadlock detection: deadlocks can be described in terms of a directed graph called a wait-for graph. This graph consists of a pair $G = (V, E)$ where V is a set of vertices & E is a set of edges. The vertices represent x'actions in the system. Each edge is an ordered pair $T_i \rightarrow T_j$. If $T_i \rightarrow T_j$ is in E then it implies that x'action T_i is waiting for T_j .

A deadlock exists if & only if the wait for graph contains a cycle. Each x'action involved in the cycle is said to be deadlocked.

To detect deadlock, the system has to maintain wait for graph & periodically to invoke an algorithm that searches for a cycle in the graph.

Since there is no cycle in this graph, the deadlock has not occurred.



Because of cyclic wait among x'actions T_{26} , T_{27} & T_{28} . Now system is in deadlock state.

Recovery from deadlock:

When deadlock is detected, system has to rollback one or more x'actions to break the cycle in the wait for graph.

Three actions are taken.

(i) Selection of victim: Given a set of deadlocked x'actions system has to determine which x'action to rollback to break the cycle. System should select the x'action that will incur the minimum cost.

There are many factors, affecting cost of a x'action rollback.

How long x'action has done & how longer it will continue.

How many data items the x'action has used.

How many more data items the x'action needs.

How many x'actions will be involved in the rollback.

(ii) Rollback: Once a x'action is selected system has to determine how far this x'action should be rolled back.

Total roll back: Abort the x'action completely & restart it from the beginning.

Partial roll back: x'action is rolled back only as far as necessary to break the cycle.

(iii) Starvation: In a system where x'action is selected on the basis of only cost, it may happen that the same x'action is always selected as victim. So, this x'action never completes its task & so there is starvation.

System must ensure that x'action can be selected as a victim only a small finite no. of times.

[Q: What is a deadlock? What are the conditions necessary for deadlocks? Discuss the various strategies used for handling deadlocks. May08]

[Q: What is the effect of Deadlock on DBMS? How it is detected? Discuss different types of deadlock avoidance schemes. May08]

[Q: Explain the preemptive and non preemptive method used for deadlock prevention and compare the two methods. May08]

[Q: Explain Deadlock Handling in database system. May08]

